SDP

MCP Apps

Rich Interactive UI in Chat

Transform chat from text stream to visual workspace with inline charts, tables, and forms

GitHub Copilot · Model Context Protocol · VS Code

🤔 The Core Question

"What if your chat responses were interactive visualizations you could explore inline?"

Chat returns text. MCP Apps return
charts, tables, forms, and trees — interactive, inline, no context switch.

MCP Server Author

Build tools that return component specs instead of text — VS Code renders them automatically

Data-Heavy Developer

Replace copy-to-Excel workflows with inline sortable tables and drill-down charts

Tool Builder

Compose custom agent workflows with stateful multi-step form interactions in chat

12 min → 45 sec

Copy-to-Excel chart workflow collapses to inline bar chart — 90 min/day reclaimed in data-heavy workflows

6 component types

Charts, tables, forms, trees, cards, custom — one decision tree maps any use case to the right type

type: component

Return one field differently in your MCP tool response — VS Code renders an interactive iframe in chat

Table of Contents

Click any section to jump directly there

PART 1

Component Types

Six built-in component types cover the full visualization surface — one decision tree maps any use case.

Charts & Tables

Visual data with hover, zoom, sort, filter, export

Forms & Trees

Structured input, hierarchical navigation with callbacks

Cards & Custom

Rich layouts and sandboxed HTML for anything else

```
Ask: show sales data
↳ Interactive bar chart renders inline — filter, hover, drill down
```

Six Component Types — One Decision Tree

chart	Time-series, comparisons, distributions — bar, line, pie, scatter, area	`visual data`
table	>20 rows needing sort, filter, search, pagination, CSV export	`tabular data`
form	Multi-field structured input with validation, dropdowns, callbacks	`user input`
tree	File systems, org charts, nested categories — expand/collapse + select	`hierarchical`
cards	Options, portfolios, galleries — grid/list/carousel with action buttons	`rich layout`
custom	Sandboxed HTML/CSS/JS for flame graphs, maps, or any bespoke visual	`anything else`

Default to built-in types. Reach for custom only when no built-in type fits.

Charts and Tables — The Workhorses

Charts

Five chart types
bar, line, pie, scatter, area — all interactive

Hover, zoom, drill-down
Mouse over for exact values; click for details

Custom colors + legend
Adapts to VS Code theme variables automatically

Tables

Sort and filter
Click column headers; dropdown filters for enum columns

Full-text search
Search box across all columns, instant results

Pagination + CSV export
10/25/50/100 per page; download button always visible

Forms, Trees, Cards — Stateful Interaction

Forms

Collect structured input with validation and submit callbacks

- text, email, number, select, checkbox
- Pattern matching + error messages
- onSubmit calls specified MCP tool

Trees

Navigate hierarchical data with expand/collapse and selection

- Lazy-load children on expand
- File-type icons or custom icons
- onSelect triggers MCP tool callback

Cards

Present options or galleries with rich layouts and actions

- Grid, list, or carousel layout
- Title, subtitle, image, badges
- Multiple action buttons per card

PART 2

Building MCP Apps

One field changes everything: return type: 'component' instead of text — VS Code renders an interactive iframe.

The Pivotal Moment

type: component in your content array triggers VS Code to render UI

Callback Lifecycle

User clicks → VS Code calls MCP tool → server returns updated state

Security Model

Sandboxed iframe, CSP restrictions, explicit callback approval

```
return { content: [{ type: "component", component: { type: "chart" ... } }]
}
↳ VS Code detects, instantiates, renders — no extra configuration
```

One Field — From Text String to Interactive Chart

```
// Before: plain text response
return {
  content: [{
    type: "text",
    text: "Jan: 45000, Feb: 52000"
  }]
};

// After: interactive chart
return {
  content: [{
    type: "component",
    component: {
      type: "chart",
      chartType: "bar",
      title: "Monthly Revenue",
      data: [
        { label: "Jan", value: 45000 },
        { label: "Feb", value: 52000 }
      ],
      options: { interactive: true }
    }
  }]
```

🔍 VS Code detects it

type: component in the content array triggers the chat renderer — no other config needed

📦 Sandboxed iframe

Component renders in an isolated iframe with VS Code theme variables available

🎨 Theme-aware

var(--vscode-editor-background) and all theme tokens accessible in custom components

The Full Interaction Loop — Stateful Without a New Prompt

User prompt	Developer types or agent routes a request that matches a tool	`chat input`
Model selects	Model identifies the right MCP tool and calls it with arguments	`tool call`
Server runs	MCP server executes logic (fetch DB, query API) and builds component spec	`server side`
VS Code renders	Chat renderer detects type: component and instantiates interactive element	`iframe render`
User interacts	Click, filter, submit — action fires; VS Code calls the registered callback tool	`user action`
Updated state	Callback tool processes data and returns a new component or text confirmation	`new response`

Callback Pattern — Form Collects, Tool Processes

```
// Tool 1: return form with callback
return {
  content: [{
    type: "component",
    component: {
      type: "form",
      title: "Create User",
      fields: [
        { name: "username", type: "text", required: true },
        { name: "role", type: "select",
          options: ["Admin","Editor","Viewer"] }
      ],
      onSubmit: "process-user-creation"
    }
  }]
};

// Tool 2: onSubmit callback
if (name === "process-user-creation") {
  const { username, role } = args;
  await createUser(username, role);
  return { content: [{ type: "text",
    text: `User ${username} created!` }] };
```

📋 onSubmit names the tool

String value is the exact MCP tool name VS Code will call with the form data as arguments

🌲 onSelect for trees

Tree node clicks call the specified tool with the node's data payload as arguments

✅ Explicit approval

VS Code shows approval prompt before invoking callback tool — user controls what executes

Security by Default — Sandboxed, Restricted, Explicit

📦

Sandboxed iframe

Custom components run in isolated iframes — no access to VS Code APIs or the host page DOM

🛡️

CSP restrictions

Content Security Policy prevents arbitrary external requests from component scripts

✋

Explicit callbacks

VS Code prompts for approval before invoking any MCP tool triggered by component interaction

🎨

Theme tokens only

Custom components access VS Code color tokens via CSS variables — nothing else from the host

💡 Use var(--vscode-editor-background) and related CSS variables in custom components — they auto-adapt to light and dark themes.

PART 3

Real-World Patterns

Four canonical scenarios that move from 'can build' to 'builds well' — with the 12-min → 45-sec payoff.

Dashboard

Multi-component overview from a single query

Drill-Down

Chart click opens detail table — no new prompt

Form Workflow

Multi-step guided collection with validation and callbacks

```
Manual: copy → Excel → chart → 12 minutes
↳ MCP Apps: same query → 45 seconds inline
```

Dashboard — Multi-Component Response from One Query

```
// One tool call returns chart + table together
return {
  content: [
    { type: "component", component: { type: "chart",
      chartType: "bar", title: "Commits by Author", data: authorStats } },
    { type: "component", component: { type: "table",
      title: "Recent Pull Requests",
      columns: [{ key: "title", sortable: true }, { key: "status", filterable: true }],
      data: recentPRs, options: { pagination: true } } }
  ]
};
```

🔄 Multiple components in one response

Return array with chart + table + cards together — comprehensive view from a single query

🧩 Each component independent

Sort the table, hover the chart — components interact independently with their own callbacks

⚡ No context switch

Entire monitoring dashboard inline — developer never leaves chat to build the same view externally

Progressive Drill-Down — Summary to Detail Without a New Prompt

📈 Step 1: Summary chart

Tool returns bar chart
Sales by region — overview at a glance

onClick: show-region-details
Click any bar to drill into that region

Context preserved
Original chart stays visible above the detail

📋 Step 2: Detail table

Callback receives region name
Arguments from the clicked bar — no prompt needed

Returns sortable detail table
Product, units, revenue — exportable to CSV

Natural master-detail flow
Same UX as a native app — stays inside chat

12 Minutes of Manual Work → 45 Seconds Inline

Copy-to-Excel workflows collapse — in data-heavy roles, that's 90 minutes per day reclaimed

16×

faster than copy-export-chart

12 min manual → 45 sec inline — repeated dozens of times per day

Form workflow pattern

Multi-step guided data collection with validation — no external form builder needed

Tree navigation pattern

File system exploration with inline previews on select — stays in chat throughout

MCP Apps Playground

Working examples of all four patterns available as a hands-on follow-up resource

The Playground repository has runnable implementations of all four canonical patterns — import and adapt, no blank-page start.

Build Well — Five Rules for Production MCP Apps

Component-first	Return UI for visual data — never ASCII charts or 500-row markdown tables	`default to UI`
Progressively	Show summary first; enable drill-down — manage context window while preserving depth	`summary → detail`
Callback-driven	Let user interactions call back to MCP tools — stateful workflow without new prompts	`stateful`
Theme-aware	Use var(--vscode-*) tokens in custom components — adapts to light and dark mode	`CSS variables`
Sandbox always	Never return unsandboxed HTML/JS — XSS risk, CSP violations, theme inconsistency	`iframe only`

PART 4

Integration

MCP Apps become first-class platform capability in agentic workflows — not a feature, a platform.

Custom Agents

Wire MCP App tools into agent definitions for visual-first workflows

Agent Skills

Skills declare which dashboard tools they use — composable visualization

Copilot Memory

Persist chart preferences and UI state across sessions

```
agent calls analytics-app.show-metrics automatically
↳ Interactive chart in chat — no manual tool invocation needed
```

Agents and Skills — Visual-First Agentic Workflows

Custom Agents

Declare MCP server in agent def

mcp-servers: analytics-app wires all its tools into the agent

Agent uses tools automatically

"Analyze last month's sales" → agent calls show-metrics → chart renders

No manual tool invocation

Developer describes intent — agent selects and calls the right component tool

Agent Skills

Skills list which tools they use

review-dashboard/show-complexity — explicit tool declarations

Composable visualization

Code review skill calls coverage table + complexity chart as workflow steps

Shareable patterns

Org skill library can publish visual reporting skills for all teams

Copilot Memory — Persistent UI State Across Sessions

📌

Store Preferences

User says "remember I prefer bar charts over pie" — Memory stores it, every future query uses bar charts automatically

🔄

Persistent State

Dashboard filter settings, selected regions, and drill-down context survive session restarts — pick up where you left off

🏢

Team Conventions

Organization-level Memory can store chart color palettes and table column preferences — visual consistency across the team

MCP Apps as Platform — Not Feature, Architecture

Build once

MCP App tools are composable — dashboard used standalone or wired into agents and skills

Agent-native

Agents call visualization tools the same way they call any MCP tool — charts appear automatically

Org-wide library

Share visualization skills in the org skill library — every team gets the same interactive reporting

Memory-backed state

Preferences, filters, and context persist — interactive UX that feels like a native application

🚀 The progression: standalone MCP App → agent-integrated tool → org skill → Memory-backed preference. Each step multiplies reach.

The Value Recap

From Text-Stream Chat to Visual Workspace

Before

- Data responses are markdown tables or ASCII — unreadable beyond toy examples
- Copy data to Excel, build chart, realize wrong date range, repeat — 12 min per query
- Static results: re-prompt and start over when you need a different filter or view
- Context switches accumulate: 90 minutes per day lost to data formatting in heavy workflows

After

- Charts, tables, forms, and trees render inline — interactive from the moment they appear
- Click to filter, hover for details, drill down into regions — no new prompts needed
- Callback-driven state: form submissions and tree selections call MCP tools and return updates
- 12 minutes of manual copy-export-chart work → 45 seconds inline — flow preserved throughout

16×

faster than copy-to-Excel chart workflow — 12 min manual → 45 sec inline

90 min/day

reclaimed in data-heavy workflows from eliminating context-switch overhead

6 types

charts, tables, forms, trees, cards, custom — one decision tree maps any use case

✅ What You Can Do Today

Today

- Install the MCP Apps Playground and run all six component type examples locally
- Change one existing MCP tool to return type: component instead of text — build your first chart
- Map your most text-heavy tool response to the decision tree: which component type fits best?

This Week

- Add a sortable table component to your highest-traffic data query tool
- Implement one callback: form onSubmit or tree onSelect calling a second MCP tool
- Test the security model — verify iframe sandboxing and CSP restrictions with browser devtools

This Month

- Build a dashboard tool returning chart + table from a single query
- Wire your MCP App into a Custom Agent definition so it fires automatically on data requests
- Set a Copilot Memory preference (chart type, color palette) and verify it persists across sessions

🔑 Key Takeaway

MCP Apps transform chat from a text stream into a visual workspace — start with one tool, one component type, one callback.

References

Official Documentation

MCP Apps support in VS Code
Introduction, capabilities overview, and getting started guide

MCP servers documentation in VS Code
MCP integration, configuration, and debugging reference

Model Context Protocol specification
Core MCP protocol spec — component content type details

SDKs & Examples

MCP servers repository (Playground)
Working examples of all component types — install locally for live demos

MCP TypeScript SDK
SDK for building MCP servers — types, transport, request handlers

MCP Apps

Rich Interactive UI in Chat

type: component

one field change in your MCP tool response — VS Code renders an interactive iframe automatically

16×

faster than copy-to-Excel — 12-minute chart workflow collapses to 45 seconds inline

6 types

charts, tables, forms, trees, cards, custom — composable into agents, skills, and Memory-backed workflows

Which of your existing MCP tools has the most text-heavy response that would benefit from a chart or table component?